

[[nodiscard]] in the Standard Library: Clause 23 Iterators library

Document #: P2377R0
Date: 2021-05-11
Project: Programming Language C++
Audience: Library Evolution
Reply-to: Christopher Di Bella
<cjdb.ns@gmail.com>

Contents

- 1 Abstract
- 2 Change log
 - 2.1 R0
- 3 Motivation
 - 3.1 Why wasn't this done in C++20?
 - 3.2 Should this be backported to earlier C++ standards?
- 4 Design
 - 4.1 Customisation points
 - 4.2 Iterator operations [iterator.operations] and [range.iter.ops]
 - 4.2.1 Advance
 - 4.2.2 Distance
 - 4.2.3 Next and prev
 - 4.3 Predefined iterators and sentinels
 - 4.3.1 Reverse iterator
 - 4.3.2 Insert iterators
 - 4.3.3 Move iterators and sentinels
 - 4.3.4 Range access
- 5 Future work
 - 5.1 Why this is worth the committee's time
- 6 Proposed wording
 - 6.1 Customization point object types [customization.point.object]

- 6.2 Header `<iterator>` synopsis [iterator.synopsis]
- 6.3 Class template `reverse_iterator` [reverse.iterator]
- 6.4 Class template `back_insert_iterator` [back.insert.iterator]
- 6.5 Class template `front_insert_iterator` [front.insert.iterator]
- 6.6 Class template `insert_iterator` [insert.iterator]
- 7 Class template `move_iterator` [move.iterator]
 - 7.1 Class template `move_sentinel` [move.sentinel]
 - 7.2 Class template `common_iterator` [common.iterator]
 - 7.3 Class template `counted_iterator` [counted.iterator]
 - 7.4 Class template `istream_iterator` [istream.iterator]
 - 7.5 Class template `ostream_iterator` [ostream.iterator]
 - 7.6 Class template `istreambuf_iterator` [istreambuf.iterator]
 - 7.7 Class template `ostreambuf_iterator` [ostreambuf.iterator]
- 8 Acknowledgements

§ 1 Abstract

During NB ballot resolution for C++20, GB254 and GB271 requested that certain ranges operations be marked `[[nodiscard]]` to indicate their return values are the salient focus of the call. Library Evolution rejected this request because “of the specification complexity for the CPOs in question it isn’t clear *where* to add `[[nodiscard]]`. This can be done later with a paper that resolves that wording issue. No consensus for change in C++20”.

This paper revisits the issue for the whole of the iterators library.

§ 2 Change log

§ 2.1 R0

Initial paper.

§ 3 Motivation

C++’s defaults are often backwards. We have `explicit` instead of `implicit`, `noexcept` instead of `throws`, `const` instead of `mutable`, and `[[nodiscard]]` instead of `[[discardable]]`, to name a few. Humans often make mistakes (and costly ones at that), so it’s good to turn these sorts of things on whenever they make sense to do so. For

example, `std::ranges::empty` may be interpreted to be a `clear` operation, because it lacks an `is_` prefix. Thus, the expression-statement `std::ranges::empty(r);` is problematic, because it's not doing what the user intended. The author is of the opinion that *any* operations where the salient feature is its return value fall into this category, regardless of whether or not the operation should be prefixed with something like `is_`. For example, `std::ranges::next(i);` is another expression-statement that's as smelly as `std::ranges::empty(r);`: why is the result of a modified copy being discarded? The result should either be used or the operation shouldn't be called.

§ 3.1 Why wasn't this done in C++20?

C++20 NB comment GB254 requested:

Most of the ranges iterator operations should be marked `[[nodiscard]]`. These are equality-preserving operations that return values always intended to be used. The library should reflect this.

Proposed change: Add `[[nodiscard]]` to the following operations.

- `ranges::iter_move`
- `ranges::distance`
- `ranges::next`
- `ranges::prev`

The BSI C++ panel has a policy requiring each NB comment to be co-sponsored, and mustn't be controversial. Due to the volume of NB comments, and the fact that it wasn't clear what some of the wording should look like, LEWG issued the following response.

LEWG in Belfast: Because of the specification complexity for the CPOs in question it isn't clear *where* to add `[[nodiscard]]`. This can be done later with a paper that resolves that wording issue. No consensus for change in C++20.

§ 3.2 Should this be backported to earlier C++ standards?

The author recommends that implementers apply `[[nodiscard]]` to relevant sections for C++17 and C++20.

§ 4 Design

Not everything in the standard library should be marked `[[nodiscard]]`. We'll consider every operation, and discuss whether or not it's a good idea to slap the attribute on the operation's side. For convenience, we use the term *pure* to mean that using the result of the operation is the sole purpose of using it, and that either the result should be used or the

operation shouldn't happen: these functions should be marked `[[nodiscard]]`. We use the term *side effect* to indicate that the focus isn't on a return value (if one exists); and while the result may be interesting, its result mightn't be useful, and thus shouldn't be specified as `[[nodiscard]]`.

§ 4.1 Customisation points

- `ranges::iter_move` is *pure* (despite the fact it moves).
- `ranges::iter_swap`'s return type is `void`, which doesn't need to be discarded.

It should be noted that the wording for customisation point objects is very different from other invocables in the standard library. Most invocables are either specified as functions or explicitly as function objects. While CPOs are function objects, their definitions are much more fuzzy, and there's no "interface" to apply a `[[nodiscard]]` to.

The author can think of two ways in which we can apply the attribute.

One way is to simply make *expression-statements* equivalent to `ranges::iter_move(E);` ill-formed, no diagnostic required. This implies a `[[nodiscard]]` attribute is applicable: a diagnostic *may* be issued, or the diagnostic may be turned off. A potential issue with this approach is that LEWG will need to consider this for every non-void returning CPO, including all range adaptor closure objects. This doesn't scale well.

Another way is to add blanket wording to `[customization.point.object]`, saying something along the lines of

Unless otherwise specified, if the return type of a function call operator is not `void`, then its *attribute-specifier-seq* shall include `[[nodiscard]]`.

This blanket wording, if adopted, means that `ranges::begin`, etc., won't need to be addressed in *[[nodiscard]] in the Standard Library: Clause 24 Ranges library*. It also covers any future CPOs that we might introduce. It also explicitly indicates that the `[[nodiscard]]` is to be applied to a call operator, and offers escape hatches for both `void`-returning call operators and CPOs where the return value might not always be interesting. Given these reasons, the author favours this approach over the IFNDR one.

§ 4.2 Iterator operations `[iterator.operations]` and `[range.iter.ops]`

§ 4.2.1 Advance

- `std::advance`, `std::ranges::advance(i, n)`, and `std::ranges::advance(i, s)`'s return types are `void`, which doesn't need to be discarded.
- `std::ranges::advance(i, n, s)` has a *side effect*.

§ 4.2.2 Distance

`std::distance` and both overloads for `std::ranges::distance` are *pure*.

§ 4.2.3 Next and prev

`std::(ranges::)?next` and `std::(ranges::)?prev` copy an iterator, advance it, and then return the advanced iterator by value. These are *pure*.

§ 4.3 Predefined iterators and sentinels

- All comparisons are *pure*.
- All `operator*`s, `operator[]`s, and `operator->`s are *pure*.
- All assignment and compound assignment operations have *side effects*.
- All increment and decrement operations have *side effects*.
- All `operator++`s and `operator--`s are *pure*.
- All `const-` and reference-qualified member functions are *pure*.
- All `iter_move` friends are *pure* (despite the fact they move).

Despite the fact that post-increment and post-decrement operations have *side effects*, the author believes that non-void post-increment and post-decrement operations should communicate to the reader that the result is intended to be used (and thus should be marked `[[nodiscard]]`). However, after around eight years of teaching and code reviews, the author isn't convinced that this viewpoint is standard practice.

§ 4.3.1 Reverse iterator

- `std::make_reverse_iterator` is *pure*.

§ 4.3.2 Insert iterators

- `std::back_inserter`, `std::front_inserter`, and `std::inserter` are *pure*.

§ 4.3.3 Move iterators and sentinels

- `std::make_move_iterator` is *pure*.

§ 4.3.4 Range access

All of `c/r?begin`, `c/r?end`, `s?size`, `empty`, and `data` are *pure*.

§ 5 Future work

If Library Evolution has an appetite for this paper, then the author will readily pen similar papers for the ranges library, the algorithms library, and the some of the general utilities library. The author is willing to also assess the containers and string libraries. Other

sections of the standard library should probably be addressed by folks with more experience using those sections.

§ 5.1 Why this is worth the committee's time

This proposal, and any that follow it, are worth the committee's time is because it's the right thing to do. We have an obligation to provide a quality standard, and ensuring this attribute is applied where it's relevant improves the quality of the document.

Microsoft/STL already aggressively pursues adding `[[nodiscard]]` to things that aren't explicitly marked as such in the standard, but `libc++` and `libstdc++` don't seem to have it in anywhere near as many places (a case-insensitive search for both `nodiscard` and `warn_unused_result` was conducted). This could be regarded as a quality-of-implementation issue, with MSVC clearly being in the lead, but since this is interface-related, we have the opportunity to unilaterally improve all implementations. By adding `[[nodiscard]]` to the synopses, we remind implementers that they should consider adding it to their respective implementations, which is similar to how Clang, GCC, and MSVC treat an unused result when it's marked with `[[nodiscard]]`. Implementations are free to disregard a placed attribute, but doing so becomes an active decision regarding QoI, rather than a possible human oversight that can be easily missed in the code review stage.

A secondary argument is that although the primary audience for the International Standard are the implementers (who often partake in authoring it), they are not the only audience. A key secondary audience to consider in this proposal's space are people who translate parts of the International Standard into material that's more digestible for the average C++ programmer: online documentation authors, book authors, teachers, and so on. While committee members seem to be aware that implementers can choose to disregard what the standard has to say about attributes, the author hasn't been able to find any explicit wording outlining this. Instead, it seems to be implied by the *as if* rule, since most attributes do not change the meaning of a program. This isn't a trivial property, and while the primary audience knows this, this secondary educational audience can be forgiven for not being aware of it.

This audit only needs to happen once for each clause, if Library Evolution ensures that `[[nodiscard]]` is considered during the design review. We don't need to dedicate lots of time to this effort either. We could set up a task force consisting of interested peoples; once a clause has been audited, it can be forwarded to Library Evolution for a brief review, mostly as a formality.

§ 6 Proposed wording

To avoid repetition, the proposed wording only concerns itself with synopses. Since `[[nodiscard]]` is really only necessary on one declaration, it's left up to the editors to decide whether or not each section should apply `[[nodiscard]]` too.

§ 6.1 Customization point object types [customization.point.object]

...

- 5 Each customization point object type constrains its return type to model a particular concept.
- 6 Unless otherwise specified, if the return type of a function call operator is not void, then its attribute-specifier-seq shall include `[[nodiscard]]`.

...

§ 6.2 Header <iterator> synopsis [iterator.synopsis]

```
// [iterator.operations], iterator operations
template<class InputIterator, class Distance>
    [[nodiscard]] constexpr
    advance(InputIterator& i, Distance n);
template<class InputIterator>
    [[nodiscard]] constexpr typename
        iterator_traits<InputIterator>::difference_type
    distance(InputIterator first, InputIterator last);
template<class InputIterator>
    [[nodiscard]] constexpr InputIterator
    next(InputIterator x,
        typename iterator_traits<InputIterator>::difference_type n = 1);
template<class InputIterator>
    [[nodiscard]] constexpr InputIterator
    prev(InputIterator x,
        typename iterator_traits<InputIterator>::difference_type n = 1);

// [range.iter.ops], range iterator operations
namespace ranges {
    ...
    // [range.iter.op.distance], ranges::distance
    template<input_or_output_iterator I, sentinel_for<I> S>
        [[nodiscard]] constexpr iter_difference_t<I> distance(I first, S
            last);
    template<range R>
        [[nodiscard]] constexpr range_difference_t<R> distance(R&& r);

    // [range.iter.op.next], ranges::next
    template<input_or_output_iterator I>
        [[nodiscard]] constexpr I next(I x);
    template<input_or_output_iterator I>
        [[nodiscard]] constexpr I next(I x, iter_difference_t<I> n);
    template<input_or_output_iterator I, sentinel_for<I> S>
        [[nodiscard]] constexpr I next(I x, S bound);
    template<input_or_output_iterator I, sentinel_for<I> S>
        [[nodiscard]] constexpr I next(I x, iter_difference_t<I> n, S bound);

    // [range.iter.op.next], ranges::next
```

```

template<bidirectional_iterator I>
    [[nodiscard]] constexpr I prev(I x);
template<bidirectional_iterator I>
    [[nodiscard]] constexpr I prev(I x, iter_difference_t<I> n);
template<bidirectional_iterator I>
    [[nodiscard]] constexpr I prev(I x, iter_difference_t<I> n, S bound);
}

// [predef.iterators], predefined iterators and sentinels
// [reverse.iterators], reverse iterators
template<class Iterator> class reverse_iterator;

template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator==(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator!=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator<(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator>(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator<=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator>=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y);
template<class Iterator1, three_way_comparable_with<Iterator1> Iterator2>
    [[nodiscard]] constexpr compare_three_way_result_t<Iterator1, Iterator2>
        operator<=>(const reverse_iterator<Iterator1>& x,
            const reverse_iterator<Iterator1>& y);

template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr auto operator-(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator1>& y) -> decltype(y.base() -
            x.base());
template<class Iterator>
    [[nodiscard]] constexpr reverse_iterator<Iterator> operator+(
        iter_difference_t<Iterator> n,
        const reverse_iterator<Iterator>& x);

template<class Iterator>
    [[nodiscard]] constexpr reverse_iterator<Iterator>
        make_reverse_iterator(Iterator i);

// [insert.iterators], insert iterators

```

```

template<class Container> class back_insert_iterator;
template<class Container>
    [[nodiscard]] constexpr back_insert_iterator<Container>
        back_inserter(Container& x);

template<class Container> class front_insert_iterator;
template<class Container>
    [[nodiscard]] constexpr front_insert_iterator<Container>
        front_inserter(Container& x);

template<class Container> class insert_iterator;
template<class Container>
    [[nodiscard]] constexpr insert_iterator<Container>
        inserter(Container& x, ranges::iterator_t<Container> i);

// [move.iterators], move iterators and sentinels
template<class Iterator> class move_iterator;

template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator==(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator!=(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator<(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator>(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator<=(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator1>& y);
template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr bool operator>=(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator1>& y);
template<class Iterator1, three_way_comparable_with<Iterator1> Iterator2>
    [[nodiscard]] constexpr compare_three_way_result_t<Iterator1, Iterator2>
        operator<==(const move_iterator<Iterator1>& x,
                    const move_iterator<Iterator1>& y);

template<class Iterator1, class Iterator2>
    [[nodiscard]] constexpr auto operator-(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator1>& y)
        -> decltype(y.base() - x.base());
template<class Iterator>
    [[nodiscard]] constexpr move_iterator<Iterator>
        operator+(iter_difference_t<Iterator> n, const
            move_iterator<Iterator>& x);

```

```

template<class Iterator>
    [[nodiscard]] constexpr move_iterator<Iterator>
        make_move_iterator(Iterator i);

...

// [stream.iterators], stream iterators
template<class T, class charT = char, class traits = char_traits<charT>,
        class Distance = ptrdiff_t>
    class istream_iterator;
template<class T, class charT, class traits, class Distance>
    [[nodiscard]] bool operator==(const
        istream_iterator<T, charT, traits, Distance>& x,
                                const
        istream_iterator<T, charT, traits, Distance>& y);

template<class T, class charT = char, class traits = char_traits<charT>,
        class Distance = ptrdiff_t>
    class ostream_iterator;

template<class charT = char, class traits = char_traits<charT>>
    class istreambuf_iterator;
template<class T, class charT, class traits, class Distance>
    [[nodiscard]] bool operator==(const istreambuf_iterator<charT, traits>&
        a,
                                const
        istreambuf_iterator<charT, traits>& b);

template<class T, class charT = char, class traits = char_traits<charT>,
        class Distance = ptrdiff_t>
    class ostreambuf_iterator;

// [iterator.range], range.access
template<class C> [[nodiscard]] constexpr auto begin(C& c) ->
    decltype(c.begin());
template<class C> [[nodiscard]] constexpr auto begin(const C& c) ->
    decltype(c.begin());
template<class C> [[nodiscard]] constexpr auto end(C& c) ->
    decltype(c.end());
template<class C> [[nodiscard]] constexpr auto end(const C& c) ->
    decltype(c.end());
template<class T, size_t N> [[nodiscard]] constexpr T* begin(T (&array)
    [N]) noexcept;
template<class T, size_t N> [[nodiscard]] constexpr T* end(T (&array)[N])
    noexcept;
template<class C> [[nodiscard]] constexpr auto cbegin(const C& c)
    noexcept(noexcept(std::begin(c)))
    -> decltype(std::begin(c));
template<class C> [[nodiscard]] constexpr auto cend(const C& c)
    noexcept(noexcept(std::end(c)))
    -> decltype(std::end(c));
template<class C> [[nodiscard]] constexpr auto rbegin(C& c) ->
    decltype(c.rbegin());
template<class C> [[nodiscard]] constexpr auto rbegin(const C& c) ->
    decltype(c.rbegin());
template<class C> [[nodiscard]] constexpr auto rend(C& c) ->
    decltype(c.rend());

```

```

template<class C> [[nodiscard]] constexpr auto rend(const C& c) ->
    decltype(c.rend());
template<class T, size_t N> [[nodiscard]] constexpr reverse_iterator<T*>
    begin(T (&array)[N]) noexcept;
template<class T, size_t N> [[nodiscard]] constexpr reverse_iterator<T*>
    end(T (&array)[N]) noexcept;
template<class E> [[nodiscard]] constexpr reverse_iterator<const E*>
    begin(initializer_list<E> il) noexcept;
template<class E> [[nodiscard]] constexpr reverse_iterator<const E*>
    end(initializer_list<E> il) noexcept;
template<class C> [[nodiscard]] constexpr auto crbegin(const C& c)
    noexcept(noexcept(std::rbegin(c)))
    -> decltype(std::begin(c));
template<class C> [[nodiscard]] constexpr auto crend(const C& c)
    noexcept(noexcept(std::rend(c)))
    -> decltype(std::end(c));

template<class C> [[nodiscard]] constexpr auto size(const C& c) ->
    decltype(c.size());
template<class T, size_t N> [[nodiscard]] constexpr size_t size(const T
    (&array)[N]) noexcept;
template<class C> [[nodiscard]] constexpr auto ssize(const C& c)
    -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
template<class T, ptrdiff_t N> [[nodiscard]] constexpr ptrdiff_t
    ssize(const T (&array)[N]) noexcept;
template<class C> [[nodiscard]] constexpr auto empty(const C& c) ->
    decltype(c.empty());
template<class T, size_t N> [[nodiscard]] constexpr bool empty(const T
    (&array)[N]) noexcept;
template<class E> [[nodiscard]] constexpr bool empty(initializer_list<E>
    il) noexcept;
template<class C> [[nodiscard]] constexpr auto data(C& c) ->
    decltype(c.data());
template<class C> [[nodiscard]] constexpr auto data(const C& c) ->
    decltype(c.data());
template<class T, size_t N> [[nodiscard]] constexpr T* data(T (&array)[N])
    noexcept;
template<class E> [[nodiscard]] constexpr const E*
    data(initializer_list<E> il) noexcept;

```

§ 6.3 Class template reverse_iterator [reverse.iterator]

```

namespace std {
    template<class Iterator>
    class reverse_iterator {
    public:
        ...
        [[nodiscard]] constexpr Iterator base() const;
        [[nodiscard]] constexpr reference operator*() const;
        [[nodiscard]] constexpr pointer operator->() const requires see below;

        constexpr reverse_iterator& operator++();
        constexpr reverse_iterator operator++(int);
        constexpr reverse_iterator& operator--();
        constexpr reverse_iterator operator--(int);
    };

```

```

    [[nodiscard]] constexpr reverse_iterator operator+(difference_type n)
        const;
    constexpr reverse_iterator& operator+=(difference_type n);
    [[nodiscard]] constexpr reverse_iterator operator-(difference_type n)
        const;
    constexpr reverse_iterator& operator-=(difference_type n);
    [[nodiscard]] constexpr unspecified operator[](difference_type n)
        const;

    [[nodiscard]] friend constexpr iter_rvalue_reference_t<Iterator>
        iter_move(const reverse_iterator& i) noexcept(see below);
    ...
};
}

```

§ 6.4 Class template back_insert_iterator [back.insert.iterator]

```

namespace std {
    template<class Container>
    class back_insert_iterator {
        ...

        [[nodiscard]] constexpr back_insert_iterator& operator*();
        constexpr back_insert_iterator& operator++();
        constexpr back_insert_iterator operator++(int);
    };
}

```

§ 6.5 Class template front_insert_iterator [front.insert.iterator]

```

namespace std {
    template<class Container>
    class front_insert_iterator {
        ...

        [[nodiscard]] constexpr front_insert_iterator& operator*();
        constexpr front_insert_iterator& operator++();
        constexpr front_insert_iterator operator++(int);
    };
}

```

§ 6.6 Class template insert_iterator [insert.iterator]

```

namespace std {
    template<class Container>
    class insert_iterator {
        ...

        [[nodiscard]] constexpr insert_iterator& operator*();
        constexpr insert_iterator& operator++();
        constexpr insert_iterator& operator++(int);
    };
}

```

```
};
}
```

§ 7 Class template `move_iterator` [`move.iterator`]

```
namespace std {
    template<class Iterator>
    class move_iterator {
    public:
        ...
        [[nodiscard]] constexpr const iterator_type& base() const &;
        [[nodiscard]] constexpr iterator_type base() &&;
        [[nodiscard]] constexpr reference operator*() const;

        constexpr move_iterator& operator++();
        constexpr auto operator++(int);
        constexpr move_iterator& operator--();
        constexpr move_iterator operator--(int);

        [[nodiscard]] constexpr move_iterator operator+(difference_type n)
            const;
        constexpr move_iterator& operator+=(difference_type n);
        [[nodiscard]] constexpr move_iterator operator-(difference_type n)
            const;
        constexpr move_iterator& operator-=(difference_type n);
        [[nodiscard]] constexpr reference operator[](difference_type n) const;

        template<sentinel_for<Iterator> S>
        [[nodiscard]] friend constexpr bool
            operator==(const move_iterator& x, const move_sentinel<S>& y);
        template<sized_sentinel_for<Iterator> S>
        [[nodiscard]] friend constexpr iter_difference_t<Iterator>
            operator==(const move_sentinel<S>& x, const move_iterator& y);
        template<sized_sentinel_for<Iterator> S>
        [[nodiscard]] friend constexpr iter_difference_t<Iterator>
            operator==(const move_iterator& x, const move_sentinel<S>& y);
        [[nodiscard]] friend constexpr iter_rvalue_reference_t<Iterator>
            iter_move(const move_iterator& i) noexcept(see below);
        ...
    };
}
```

§ 7.1 Class template `move_sentinel` [`move.sentinel`]

```
namespace std {
    template<semiregular S>
    class move_sentinel {
    public:
        ...
        [[nodiscard]] constexpr S base() const;
        ...
    };
}
```

```
};
}
```

§ 7.2 Class template `common_iterator` [`common.iterator`]

```
namespace std {
    template<input_or_output_iterator I, sentinel_for<I> S>
        requires (!same_as<I, S> && copyable<I>)
        class common_iterator {
        public:
            ...

            [[nodiscard]] decltype(auto) operator*();
            [[nodiscard]] decltype(auto) operator*() const
                requires dereferenceable<const I>;
            [[nodiscard]] decltype(auto) operator->() const
                requires see below;

            common_iterator& operator++();
            decltype(auto) operator++(int);

            template<class I2, sentinel_for<I> S2>
                requires sentinel_for<S, I2>
            [[nodiscard]] friend bool operator==(
                const common_iterator& x, const common_iterator<I2, S2>& y);
            template<class I2, sentinel_for<I> S2>
                requires sentinel_for<S, I2> && equality_comparable_with<I, I2>
            [[nodiscard]] friend bool operator==(
                const common_iterator& x, const common_iterator<I2, S2>& y);

            template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
                requires sized_sentinel_for<S, I2>
            [[nodiscard]] friend iter_difference_t operator-(
                const common_iterator& x, const common_iterator<I2, S2>& y);

            [[nodiscard]] friend iter_rvalue_reference_t<I> iter_move(const
                common_iterator& i)
                noexcept(noexcept(ranges::iter_move(declval<const I>&())))
                requires input_iterator<I>;
            ...
        };
}
```

§ 7.3 Class template `counted_iterator` [`counted.iterator`]

```
namespace std {
    template<input_or_output_iterator I>
    class counted_iterator {
    public:
        ...
        [[nodiscard]] constexpr const I& base() const &;
        [[nodiscard]] constexpr I base() &&;
        [[nodiscard]] constexpr iter_difference_t<I> count() const noexcept;
```

```

[[nodiscard]] constexpr decltype(auto) operator*();
[[nodiscard]] constexpr decltype(auto) operator*() const
    requires dereferenceable<const I>;

[[nodiscard]] constexpr auto operator->() const noexcept
    requires contiguous_iterator<I>;

constexpr counted_iterator& operator++();
decltype(auto) operator++(int);
constexpr counted_iterator operator++(int)
    requires forward_iterator<I>;
constexpr counted_iterator& operator--()
    requires bidirectional_iterator<I>;
constexpr counted_iterator operator--(int)
    requires bidirectional_iterator<I>;

[[nodiscard]] constexpr counted_iterator operator-
    (iter_difference_t<I> n) const
    requires random_access_iterator<I>;
template<common_with<I> I2>
    [[nodiscard]] constexpr iter_difference_t<I2> operator-(
        const counted_iterator& x, const counted_iterator<I2>& y);
[[nodiscard]] constexpr iter_difference_t<I2> operator-(
    const counted_iterator& x, default_sentinel_t);
[[nodiscard]] constexpr iter_difference_t<I2> operator-(
    default_sentinel_t, const counted_iterator& y);
constexpr counted_iterator& operator--=(iter_difference_t<I> n)
    requires random_access_iterator<I>;

[[nodiscard]] constexpr decltype(auto) operator[](iter_difference_t<I>
    n) const
    requires random_access_iterator<I>;

template<common_with<I> I2>
    [[nodiscard]] friend constexpr bool operator==(
        const counted_iterator& x, const counted_iterator<I2>& y);
[[nodiscard]] friend constexpr bool operator==(
    const counted_iterator& x, default_sentinel_t);

template<common_with<I> I2>
    [[nodiscard]] friend constexpr strong_ordering operator<=(
        const counted_iterator& x, const counted_iterator<I2>& y);

[[nodiscard]] friend constexpr iter_rvalue_reference_t<I>
    iter_move(const counted_iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current)))
    requires input_iterator<I>;

...
};
}

```

§ 7.4 Class template `istream_iterator` [`istream.iterator`]

```

namespace std {
    template<class T, class charT = char, class traits = char_traits<charT>,

```

```

        class Distance = ptrdiff_t>
class istream_iterator {
public:
    ...
    [[nodiscard]] const T& operator*() const;
    [[nodiscard]] const T& operator->() const;
    istream_iterator& operator++();
    istream_iterator operator++(int);

    [[nodiscard]] friend bool operator==(const istream_iterator& i,
        default_sentinel_t);
    ...
};
}

```

§ 7.5 Class template ostream_iterator [ostream.iterator]

```

namespace std {
    template<class T, class charT = char, class traits = char_traits<charT>>
    class ostream_iterator {
    public:
        ...
        [[nodiscard]] const T& operator*() const;
        ostream_iterator& operator++();
        ostream_iterator operator++(int);
        ...
    };
}

```

§ 7.6 Class template istreambuf_iterator [istreambuf.iterator]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class istreambuf_iterator {
    public:
        ...
        [[nodiscard]] charT operator*() const;
        istreambuf_iterator& operator++();
        proxy operator++(int);
        [[nodiscard]] bool equal(const istreambuf_iterator& b) const;

        [[nodiscard]] friend bool operator==(const istreambuf_iterator& i,
            default_sentinel_t s);
        ...
    };
}

```

§ 7.7 Class template ostreambuf_iterator [ostreambuf.iterator]

```
namespace std {  
    template<class charT, class traits = char_traits<charT>>  
    class ostreambuf_iterator {  
    public:  
        ...  
        [[nodiscard]] ostreambuf_iterator& operator*() const;  
        ostreambuf_iterator& operator++();  
        ostreambuf_iterator& operator++(int);  
        [[nodiscard]] bool failed() const;  
        ...  
    };  
}
```

§ 8 Acknowledgements

The author would like to thank Nicole Mazzuca and Ben Craig for reviewing this proposal.